

AI-Powered Intelligent Document Verification System

Author: Shambhavi Jha

Project Type: Intelligent Document Processing (IDP) for Employee Onboarding

Technology Stack: Open Source AI Models + Python

1. Introduction

Employee onboarding is one of the most important processes in an organization. During onboarding, employees submit identity documents such as Aadhaar Card and PAN Card, which are manually verified by HR. HR compares information across documents, validates document authenticity, checks mandatory fields, and finally approves or rejects the onboarding request.

This manual verification process is repetitive, time-consuming, and difficult to scale when handling a large number of employees.

The objective of this project is to develop an **AI-powered Intelligent Document Verification System** that automates the complete verification workflow using open-source technologies. The system processes uploaded documents, extracts structured information, validates it against business rules, performs cross-document verification, and generates a verification report. HR remains involved only in the final approval stage.

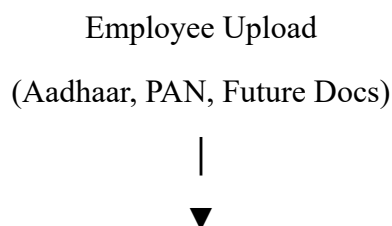
The architecture is designed to be modular so that additional document types can be supported in the future without redesigning the entire pipeline.

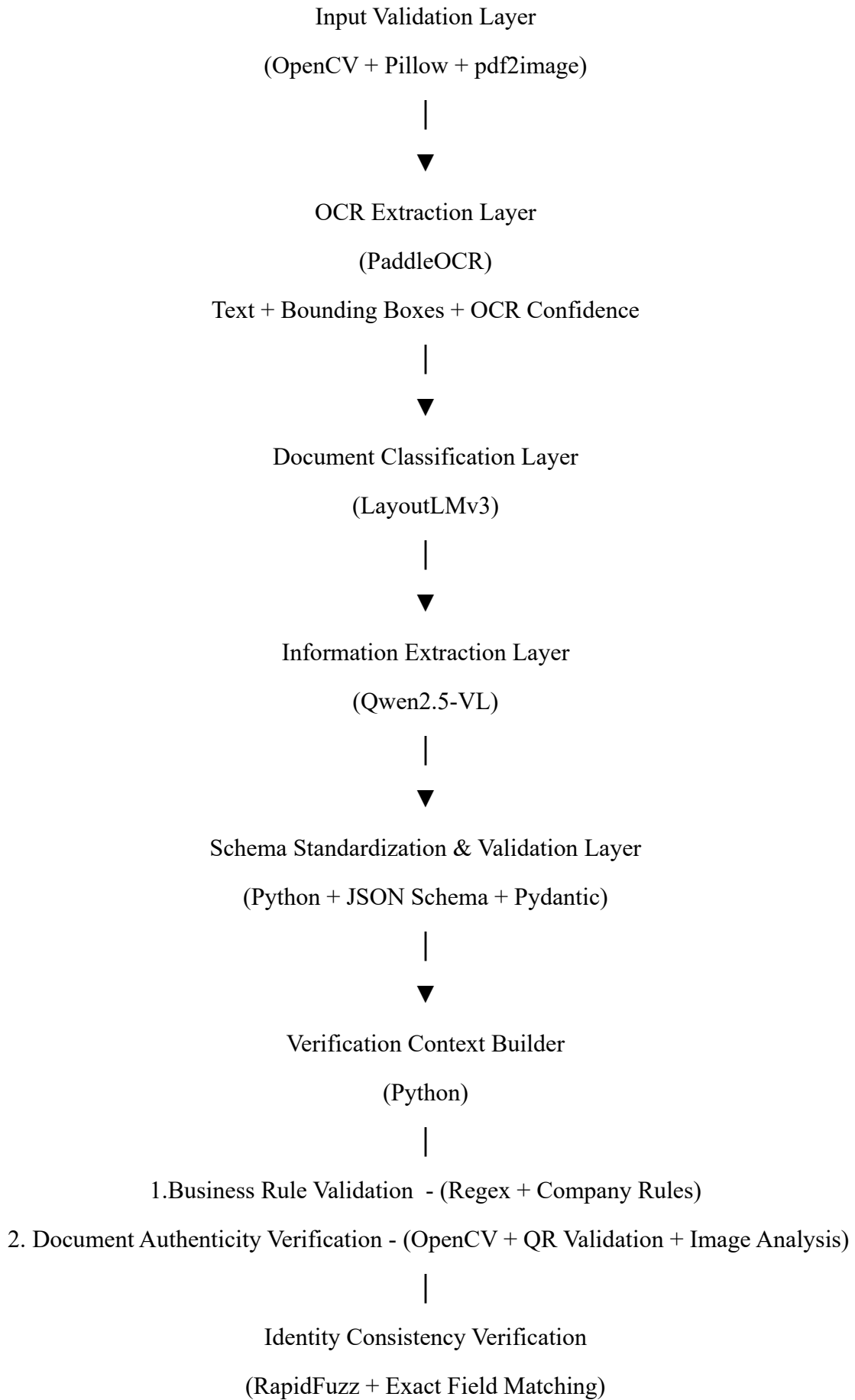
2. Objectives

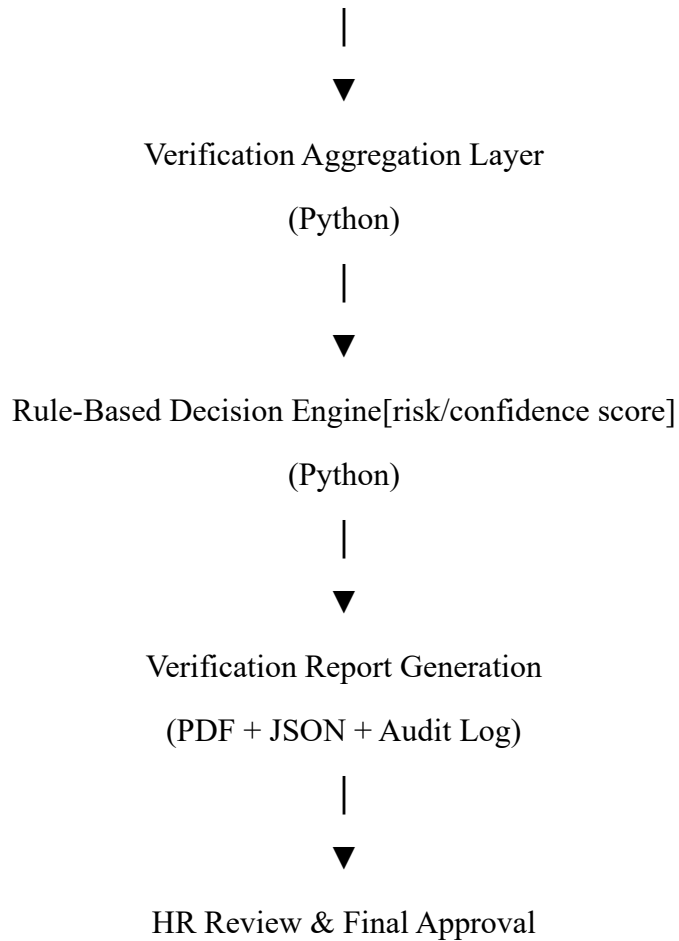
The proposed system aims to:

- Automate employee document verification.
- Reduce manual HR effort.
- Improve verification consistency.
- Minimize operational cost using open-source AI models.
- Generate explainable verification reports.
- Build a scalable architecture that supports additional document types in the future.

3. Proposed System Architecture







4. Phase-wise Implementation

Phase 1 – Input Validation Layer

Objective

Validate uploaded files before any AI processing begins.

Why is this required?

OCR and AI models perform poorly on blurred, rotated, or corrupted documents. Processing such files also increases computational cost unnecessarily.

Therefore, every uploaded document is first validated.

Input

- Image
- PDF

Processing

- File format validation
- Blur detection

- Resolution validation
- Rotation detection
- Corrupted file detection

Technology

- OpenCV
- Pillow
- pdf2image

Output

Validated Document

Phase 2 – OCR Layer

Extract textual content and spatial information from the uploaded document.

OCR converts the document into machine-readable text while also identifying the location (bounding boxes) of each detected text element. This information is essential for downstream document classification and information extraction.

Technology

PaddleOCR

Why PaddleOCR?

- Open Source and production-ready
- High OCR accuracy on Indian identity documents
- Supports multilingual text (English, Hindi, etc.)
- Returns both extracted text and bounding box coordinates
- Provides OCR confidence scores for quality assessment

Input

- Validated Image/PDF

Processing

1. Detect text regions in the document.
2. Extract textual content.
3. Generate bounding boxes for every detected text.
4. Calculate OCR confidence scores.

Output

- Raw OCR Text
- Bounding Boxes
- OCR Confidence

Example

Name : Rahul Sharma

PAN : ABCDE1234F

DOB : 12/08/1998

Phase 3 – Document Classification Layer

Automatically identify the uploaded document type.

Why is Document Classification Required?

Different identity documents follow different layouts and contain different fields.

For example:

PAN Card

- PAN Number
- Name
- Father's Name
- Date of Birth

Aadhaar Card

- Aadhaar Number
- Name
- Gender
- Address

Identifying the document type enables the system to apply the appropriate extraction schema and validation rules.

Technology

LayoutLMv3

Why LayoutLMv3?

LayoutLMv3 is a transformer model specifically designed for document understanding.

Unlike traditional image classification models, it combines three types of information:

- Document Image
- OCR Extracted Text
- Spatial Layout (Bounding Boxes)

This multimodal understanding allows the model to accurately classify structured documents even when they have different layouts or templates.

Input

- Document Image
- OCR Text (from PaddleOCR)
- Bounding Boxes (from PaddleOCR)

Processing

1. Receive OCR text and corresponding bounding boxes.
2. Analyze the document layout and textual structure.
3. Classify the uploaded document based on its visual layout and extracted content.

Output

Document Type

↓

PAN Card

or

Document Type

↓

Aadhaar Card

Phase 4 – Information Extraction Layer

Extract meaningful structured information from the classified document by interpreting the OCR output and mapping the extracted text to predefined document fields.

Why is Information Extraction Required?

PaddleOCR extracts only the textual content and its spatial location within the document. Although the document has already been classified in the previous phase, the extracted text is still unstructured.

Technology

Qwen2.5-VL

Why Qwen2.5-VL?

Qwen2.5-VL is a Vision-Language Model capable of understanding both the document context and the extracted OCR information.

It uses:

- Document Type (from LayoutLMv3)
- OCR Extracted Text (from PaddleOCR)
- Spatial Layout / Bounding Boxes (from PaddleOCR)

to identify and map each extracted text element to its corresponding business field.

Unlike OCR, which only reads text, Qwen understands the meaning of the extracted information and generates structured output.

Input

- Document Type (Phase 3)
- OCR Text (Phase 2)
- Bounding Boxes (Phase 2)

Processing Workflow

Document Type

+

OCR Extracted Text

+

Bounding Boxes

|

Semantic Document Understanding

|

Field Identification

|

Structured JSON Generation

Example

OCR Output

Rahul Sharma

ABCDE1234F

12/08/1998

Male

Document Type

PAN Card

Qwen Output

```
{  
  "document_type": "PAN",  
  "name": "Rahul Sharma",  
  "dob": "12/08/1998",  
  "gender": "Male",  
  "pan_number": "ABCDE1234F"  
}
```

Output

A standardized structured JSON containing all extracted document fields.

This JSON serves as the input for the Verification Context Builder, where information from multiple uploaded documents is combined for further validation and cross-document verification.

Phase 5 – Schema Standardization & Validation Layer

Objective

Convert the extracted information into a standardized schema that can be uniformly processed by all downstream verification modules.

Why is this required?

Although Qwen extracts structured information from the document, different document types contain different fields.

For example,

A PAN Card contains:

- PAN Number
- Name
- Father's Name
- Date of Birth

Whereas an Aadhaar Card contains:

- Aadhaar Number
- Name
- Gender
- Address

If every document keeps its own format, downstream modules such as Business Rule Validation and Cross Document Matching would need separate logic for every document type.

To solve this, all extracted information is converted into a common standardized schema before further processing.

This ensures that every downstream service receives data in a consistent format regardless of the uploaded document type.

Technology Stack

- Python
- JSON Schema
- Pydantic

Processing

1. Receive structured information extracted by Qwen2.5-VL.
2. Convert the extracted fields into a predefined JSON schema.
3. Validate the schema using Pydantic to ensure:
 - Required fields are present.
 - Data types are correct.
 - Schema structure is consistent.
4. Generate a standardized document object for downstream verification.

Example

PAN Document

```
{  
  "document_type": "PAN",  
  "name": "Rahul Sharma",  
  "dob": "12/08/1998",
```

```
"pan_number": "ABCDE1234F"
}

Aadhaar Document
{
  "document_type": "AADHAAR",
  "name": "Rahul Kumar Sharma",
  "dob": "12/08/1998",
  "gender": "Male",
  "aadhaar_number": "123412341234"
}
```

After validation, both follow a standardized schema and are ready for the Verification Context Builder.

Output

- Standardized Aadhaar JSON
- Standardized PAN JSON

Phase 6 – Verification Context Builder

Create a centralized **Verification Context** by combining all validated document schemas belonging to a single employee into one unified verification object.

Why is this required?

Until Phase 5, every uploaded document (Aadhaar, PAN, etc.) is processed independently. However, verification cannot be performed on individual documents because the remaining verification modules require access to all employee documents together.

Instead of passing multiple document JSONs separately, the system creates a centralized **Verification Context** that represents the complete onboarding case of one employee. This provides a single source of truth for all downstream verification services.

Input

- Standardized Aadhaar JSON
- Standardized PAN JSON
- Employee Session ID

Processing

1. Collect all validated document schemas generated in Phase 5.

2. Associate every document with the employee's verification session.
3. Merge all document schemas into a single **Verification Context** using Python.
4. Store employee metadata and document metadata inside the verification object.
5. Pass the Verification Context to downstream verification modules.

Technology Stack

Python

Used to merge multiple standardized document schemas into one centralized Verification Context.

Output

Verification Context

```
{  
  "employee_id": "EMP001",  
  "session_id": "VER001",  
  "documents": {  
    "aadhaar": { ... },  
    "pan": { ... }  
  }  
}
```

This Verification Context becomes the common input for all subsequent verification modules.

Phase 7 – Business Rule Validation Layer

Objective

Validate each uploaded document according to predefined business rules and company onboarding guidelines.

Why is this required?

Before comparing documents, each document must first be individually validated to ensure that it satisfies organizational onboarding policies.

This phase performs **document-level validation** and ensures that every uploaded document is structurally valid before identity comparison begins.

Input

Verification Context

Validation Rules

- PAN Number Format Validation
- Aadhaar Number Format Validation
- Mandatory Field Validation
- Missing Field Detection
- OCR Confidence Threshold
- Company HR Guidelines
- Required Document Presence

Technology Stack

- Python
- Regular Expressions (Regex)

Processing

1. Read each document from the Verification Context.
2. Apply document-specific validation rules.
3. Validate mandatory fields.
4. Validate document formats.
5. Record validation results for every document.

Output

Validation Object

PAN Format ✓

AADHAAR Format ✓

Mandatory Fields ✓

OCR Confidence 96%

Validation Status Passed

Phase 8 – Document Authenticity Verification Layer

Objective

Verify that each uploaded document is authentic and has not been tampered with before proceeding to cross-document identity verification.

Why is this required?

A document may successfully pass OCR, classification, and information extraction even if it has been edited or manipulated. For example, an attacker may modify the name, Aadhaar number, photograph, or other identity fields using image editing software.

Therefore, before comparing multiple documents, the system first verifies the authenticity of each uploaded document to detect potential tampering or inconsistencies.

Input

- Verification Context
- Original Uploaded Document
- OCR Output
- Standardized Document JSON

Technology Stack

- **OpenCV** – Image preprocessing and tampering artifact detection
- **pyzbar / OpenCV QR** – Aadhaar QR Code Extraction and Validation (if QR is available)
- **ImageHash** – Detect duplicate or suspiciously modified document images
- **Python** – Authenticity Validation Rules

Processing

1. Read each uploaded document from the Verification Context.
2. Detect image manipulation or tampering artifacts.
3. Extract and validate the Aadhaar QR Code (where available).
4. Compare QR Code information with OCR extracted information.
5. Detect inconsistencies between document content and extracted fields.
6. Generate an Authenticity Validation Result for each document.

Output

Document Authenticity Object

Document Authenticity : PASS

QR Validation : PASS

Tampering Detected : NO

Authenticity Score : 98%

or

Document Authenticity : FAILED

QR Validation : FAILED

Tampering Detected : YES

Reason :

QR data does not match OCR extracted fields.

Phase 9 – Identity Consistency Verification Layer

Objective

Verify that all uploaded identity documents belong to the same employee by comparing common identity attributes across documents.

Why is this required?

Even if every uploaded document is individually authentic, the onboarding process must ensure that all uploaded documents represent the same employee.

This phase performs multi-document identity verification by comparing shared identity attributes across Aadhaar, PAN, and future supported documents.

Input

- Verification Context
- Validation Object
- Document Authenticity Object

Technology Stack

- **RapidFuzz** – Fuzzy similarity comparison
- **Python** – Identity comparison logic

Comparison Strategy

Identity Attribute Validation Method

Name	RapidFuzz Similarity
Date of Birth	Exact Match
Gender	Exact Match
Father's Name	RapidFuzz (if available)
Address	RapidFuzz (Future Support)
PAN Number	Exact Match
Aadhaar Number	Exact Match

Processing

1. Read all employee documents from the Verification Context.
2. Compare common identity fields across documents.
3. Calculate similarity scores where applicable.
4. Detect identity mismatches.
5. Generate an Identity Consistency Score.

Output

Identity Consistency Object

Identity Consistency Score : 97%

Matched Fields : 6 / 6

Status : CONSISTENT

Phase 10 – Verification Aggregation Layer

Objective

Aggregate all verification results into a single Unified Verification Object for final decision making.

Why is this required?

Each verification module generates an independent output. Instead of allowing the Decision Engine to process multiple objects separately, this layer consolidates all verification evidence into one unified verification object.

Input

- Verification Context
- Validation Object
- Document Authenticity Object
- Identity Consistency Object
- OCR Confidence x

Technology Stack

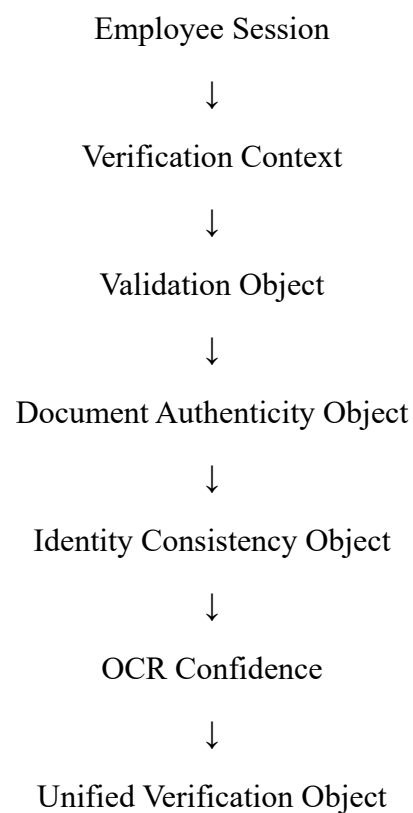
- **Python**

Processing

1. Collect outputs from all verification modules.
2. Merge all verification evidence into a Unified Verification Object.
3. Pass the unified object to the Decision Engine.

Output

Unified Verification Object



Phase 11 – Decision Engine

Objective

Generate the final verification decision using deterministic business logic.

Why is this required?

The final verification decision should be transparent, explainable, and based on predefined business rules instead of relying directly on an AI model.

Input

- Unified Verification Object

Technology Stack

- **Python Decision Engine**

Decision Logic

IF

PAN Format = Valid

AND

AADHAAR Format = Valid

AND

Document Authenticity = PASS

AND

Identity Consistency Score > 95%

AND

OCR Confidence > 90%

↓

Verified

ELSE

↓

Manual Review

↓

Rejected

Output

Verification Score : 96%

Final Status : VERIFIED

Phase 12 – Verification Report Generation

Objective

Generate a comprehensive verification report summarizing the complete verification process

Why is this required?

Instead of manually reviewing multiple documents, HR receives a structured verification report containing all validation, authenticity, and identity consistency results.

This improves transparency and significantly reduces manual verification effort.

Technology Stack

- **Python**
- **HTML / PDF Report Generator**

Report Includes

- Employee Information
- Uploaded Documents
- Extracted Information
- Business Rule Validation Results
- Document Authenticity Results
- Identity Consistency Results
- OCR Confidence
- Verification Score
- Final Decision
- Audit Summary

Output

- Verification Report (PDF)
- JSON Response
- Audit Log

Phase 13 – HR Review & Final Approval

Objective

Provide final human oversight before completing the employee onboarding process.

Why is this required?

Although the AI system automates document understanding, validation, authenticity verification, identity consistency verification, and decision support, the final onboarding approval remains with HR to ensure compliance with organizational policies.

Input

- Verification Report

HR Actions

- Approve Employee
- Reject Employee
- Request Document Re-upload
- Escalate for Manual Investigation

5. Technology Stack

Layer	Technology	Reason
Backend	FastAPI	API development and workflow integration
Input Validation	OpenCV, Pillow, pdf2image	Image preprocessing
Document Classification	LayoutLMv3	Document-aware classification using layout and text
OCR	PaddleOCR	High accuracy OCR for Indian identity documents
Information Extraction	Qwen2.5-VL	Semantic understanding and structured field extraction
JSON Standardization	Python + JSON Schema	Standard output format
Verification Context	Python + pydantic	Combines all document information into one verification session
Business Rule Engine	Python + Regex	Deterministic rule validation
Identity Consistency Verification	RapidFuzz	Efficient similarity comparison for names
Verification Aggregation	Python	Aggregates outputs from all verification modules

Layer	Technology	Reason
Document Authenticity Verification	openCv+imageHash+pyzbar	Detect potential tampering and QR inconsistencies

6. Overall Workflow

1. The employee uploads the required onboarding documents (currently Aadhaar Card and PAN Card) through the onboarding portal.
2. A unique Verification Session is created for the employee.
3. The uploaded documents undergo Input Validation.
4. PaddleOCR extracts text, bounding boxes, and OCR confidence scores.
5. LayoutLMv3 classifies the uploaded document.
6. Qwen2.5-VL extracts structured identity fields from the classified document.
7. The extracted information is standardized and validated using JSON Schema and Pydantic.
8. A centralized Verification Context is created by combining all validated document schemas.
9. The Business Rule Validation Layer validates document formats, mandatory fields, and company onboarding rules.
10. The Document Authenticity Verification Layer detects potential document tampering and validates document authenticity.
11. The Identity Consistency Verification Layer compares identity attributes across all uploaded documents to ensure they belong to the same employee.
12. The Verification Aggregation Layer consolidates all verification results into a Unified Verification Object.
13. The Decision Engine evaluates the unified verification object and generates the final verification decision (Verified, Manual Review, or Rejected).
14. A comprehensive Verification Report is generated.
15. Finally, HR reviews the verification report and performs the final onboarding action by approving, rejecting, or requesting document re-upload.

7. Conclusion

The proposed architecture presents a **modular, scalable, and cost-effective AI-powered solution** for automating employee onboarding document verification using open-source

technologies. The system combines specialized AI models with deterministic business logic to perform end-to-end document processing while minimizing manual HR intervention.

Document understanding is achieved through **PaddleOCR** for text extraction, **LayoutLMv3** for document classification, and **Qwen2.5-VL** for semantic information extraction. The extracted information is standardized into validated schemas before being consolidated into a centralized **Verification Context**.

The verification pipeline incorporates multiple validation layers, including **Business Rule Validation**, **Document Authenticity Verification**, and **Identity Consistency Verification**, ensuring that uploaded documents are structurally valid, free from potential tampering, and consistently represent the same employee. The outputs from these verification modules are consolidated by the **Verification Aggregation Layer**, after which the **Decision Engine** generates a transparent and explainable verification outcome.

By leveraging open-source AI models and Python-based validation services, the proposed system significantly reduces operational costs while maintaining scalability, explainability, and flexibility. Its modular architecture also enables seamless integration of additional document types, verification rules, and future enhancements without requiring major changes to the overall system design.